

INFORME

Integración de Arquitectura de Microservicios

Caso: Municipalidad Valle del Sol – Plataforma de Gestión de Incendios

Integrantes:

- Lucas Soto
- Joaquin Beltran

Asignatura: Fullstack III

Fecha: Junio 2026

Índice

1. Introducción	3
2. Contexto del Sistema	3
3. Descripción del Problema	3
4. Requisitos del Sistema	4
4.1 Requisitos Funcionales	4
4.2 Requisitos No Funcionales	4
5. Arquitectura del Sistema	5
5.1 Tipo de Arquitectura	5
5.2 Descripción de Capas	5
5.3 Microservicios Definidos	6
5.4 API Gateway – KrakenD	6
5.5 BFF – Backend For Frontend	6
6. Modelo de Datos	7
6.1 Modelo Relacional – NeonDB PostgreSQL	7
6.2 Modelo Documental – MongoDB Atlas	8
6.3 Seguridad JWT con RSA RS256 – MS-Usuarios	8
7. Patrones de Diseño Aplicados	9
7.1 Repository Pattern	9
7.2 Factory Method	9
7.3 Circuit Breaker – Resilience4j	9
7.4 Singleton	9
7.5 Resumen Comparativo	9
8. Decisiones Críticas de Diseño	10
9. Estrategia de Despliegue con Docker y Kubernetes	12
9.1 Docker – Construcción de Imágenes	12
9.2 Docker Compose – Ejecución Local por Microservicio	12
9.3 Kubernetes – Recursos Utilizados	12
9.4 Tabla Resumen de Componentes	13
10. Pruebas Unitarias	14
10.1 Stack de Testing por Componente	14
10.2 Cobertura de Pruebas por Componente	14
10.3 Ejemplos de Pruebas Implementadas	15
10.4 Cómo Ejecutar las Pruebas	15
11. Diagrama de Arquitectura	16
12. Consideraciones Éticas: Seguridad, Privacidad y Sostenibilidad	18
12.1 Seguridad	18
12.2 Privacidad	18
12.3 Sostenibilidad	18
13. Repositorios GitHub	18
14. Conclusión	19

1. Introducción

En los últimos años, los incendios forestales y urbanos han representado una amenaza constante para la seguridad de las personas, la infraestructura y el medio ambiente. Las municipalidades cumplen un rol fundamental en la prevención, detección temprana y coordinación de emergencias.

El presente informe documenta la solución tecnológica implementada basada en arquitectura de microservicios para la Municipalidad Valle del Sol, con el objetivo de modernizar la gestión de reportes de incendios, el monitoreo geográfico y la comunicación con la comunidad.

La arquitectura implementada integra cuatro microservicios independientes (MS-Usuarios, MS-Reportes, MS-Alertas, MS-Brigadas), un BFF en Spring Boot, un API Gateway KrakenD, y un frontend Next.js. Todo el sistema se despliega con Docker y Kubernetes, y cuenta con pruebas unitarias con cobertura superior al 60% en todos los componentes.

2. Contexto del Sistema

La Subdirección de Gestión de Emergencias y Prevención de Desastres de la Municipalidad Valle del Sol tiene como misión prevenir, detectar y coordinar situaciones de riesgo, especializándose en catástrofes de tipo forestal y urbano.

Actualmente la gestión de eventos se realiza mediante herramientas tradicionales: llamadas telefónicas, redes sociales y reportes informales. Esto genera ineficiencias críticas en la respuesta ante emergencias y una total ausencia de trazabilidad de los incidentes.

3. Descripción del Problema

El sistema actual presenta las siguientes problemáticas:

- Falta de herramientas digitales para reportar incendios de forma estructurada.
- Dificultad para visualizar la ubicación exacta de los focos de incendio en tiempo real.
- Escasa integración entre la municipalidad y organismos de emergencia como bomberos y brigadistas.
- Ausencia de mecanismos automatizados para alertar a la población cuando una emergencia escala.
- Falta de una base de datos histórica que permita analizar patrones y recurrencia.

4. Requisitos del Sistema

4.1 Requisitos Funcionales

RF1: Registrar reportes de incendios con ubicación GPS y estado.

RF2: Gestionar brigadas de emergencia (crear, actualizar estado y ubicación).

RF3: Visualizar los incendios en un mapa interactivo (MapLibre GL).

RF4: Enviar alertas a la comunidad en tiempo real.

RF5: Consultar el historial de incendios y reportes.

RF6: Gestionar usuarios con roles diferenciados (ADMIN, FUNCIONARIO, ciudadano).

RF7: Autenticación segura mediante JWT con algoritmo RSA RS256.

RF8: Enrutamiento centralizado de todas las peticiones mediante API Gateway KrakenD.

4.2 Requisitos No Funcionales

Código	Requisito	Cómo lo satisface la arquitectura
RNF1	Sistema escalable	K8s escala cada MS con réplicas automáticas según carga.
RNF2	Alta disponibilidad 24/7	MS desacoplados; el fallo de uno no tumba al resto.
RNF3	Respuesta < 3 seg.	BFF centraliza y optimiza las respuestas al frontend.
RNF4	Seguridad de datos	JWT RS256 en MS-Usuarios. Cookie HttpOnly en BFF.
RNF5	Basado en microservicios	4 MS independientes + BFF + API Gateway KrakenD, todos en K8s.
RNF6	Mantenible y modular	Repository Pattern desacopla lógica de negocio y datos.
RNF7	Integración con otros sistemas	API REST estándar. KrakenD como contrato único de integración.
RNF8	Tolerante a fallos	Circuit Breaker (Resilience4j) en BFF sobre microservicios.
RNF9	Persistencia garantizada	NeonDB (PostgreSQL serverless) y MongoDB Atlas con conexión SSL.

5. Arquitectura del Sistema

5.1 Tipo de Arquitectura

Se implementó una arquitectura de microservicios donde el sistema se divide en cuatro componentes de negocio independientes: MS-Usuarios (puerto 8081), MS-Reportes (puerto 8082), MS-Alertas (puerto 8083) y MS-Brigadas (puerto 8084), complementados por un BFF en Spring Boot (puerto 8080), un API Gateway KrakenD (puerto 8000) y un frontend en Next.js (puerto 3000). Las bases de datos son completamente gestionadas en la nube: NeonDB (PostgreSQL serverless) para MS-Usuarios, MS-Reportes y MS-Brigadas, y MongoDB Atlas para MS-Alertas.

5.2 Descripción de Capas

Capa 1 – Frontend (Next.js 15): Interfaz de usuario con React, MapLibre GL para mapas, Zustand para estado global y TypeScript. Se comunica exclusivamente con el API Gateway KrakenD.

Capa 2 – API Gateway (KrakenD): Punto único de entrada. Gestiona enrutamiento, CORS, propagación de cookies JWT y traduce rutas `/api/*` hacia el BFF. Se configura mediante `krakend.json` montado como ConfigMap en Kubernetes.

Capa 3 – BFF (Backend For Frontend): Componente Spring Boot que orquesta respuestas de múltiples microservicios. Gestiona autenticación JWT con cookie `HttpOnly` y aplica Circuit Breaker con Resilience4j.

Capa 4 – Microservicios (Pods K8s): Los cuatro MS corren como Deployments en el cluster. Se comunican con las BDs mediante Kubernetes Service DNS.

Capa 5 – Bases de datos (Nube): NeonDB (PostgreSQL serverless con SSL) y MongoDB Atlas. Completamente gestionadas en la nube, sin StatefulSets en el cluster local.

5.3 Microservicios Definidos

Microservicio	Responsabilidad	Base de datos	Patrón	RFs
---------------	-----------------	---------------	--------	-----

MS-Usuarios	Autenticación, registro y gestión de roles con JWT RS256	NeonDB (ms-usuarios)	Repository + Flyway	RF6, RF7
MS-Reportes	Registro de reportes de incendios con GPS y estado	NeonDB (ms-reportes)	Repository + Flyway + Factory Method	RF1, RF5
MS-Alertas	Gestión y envío de alertas manuales y automáticas	MongoDB Atlas	Repository + CB	RF4
MS-Brigadas	Gestión de brigadas, estado y ubicación en tiempo real	NeonDB (ms-brigadas)	Repository + Flyway + Factory Method	RF2, RF3

5.4 API Gateway – KrakenD

KrakenD es el punto único de entrada al sistema. API Gateway de alto rendimiento escrito en Go, sin estado, configurado completamente mediante krakend.json. Sus responsabilidades:

- Recibir todas las solicitudes del frontend bajo /api/* y enrutarlas al BFF (muni-bff:8080).
- Gestionar CORS centralizado, permitiendo orígenes desde el frontend y localhost:3000.
- Propagar los headers Cookie y Authorization hacia el BFF usando encoding no-op.
- Exponer todos los endpoints: /api/auth/*, /api/reportes/*, /api/alertas/*, /api/brigadas/*.
- Desplegar como Deployment en Kubernetes configurado via ConfigMap con el krakend.json.

5.5 BFF – Backend For Frontend

- Gestionar la autenticación JWT: consulta MS-Usuarios y almacena el token en cookie HttpOnly.
- Orquestrar llamadas a múltiples microservicios en una sola petición del frontend.
- Aplicar Circuit Breaker con Resilience4j sobre las llamadas a microservicios.

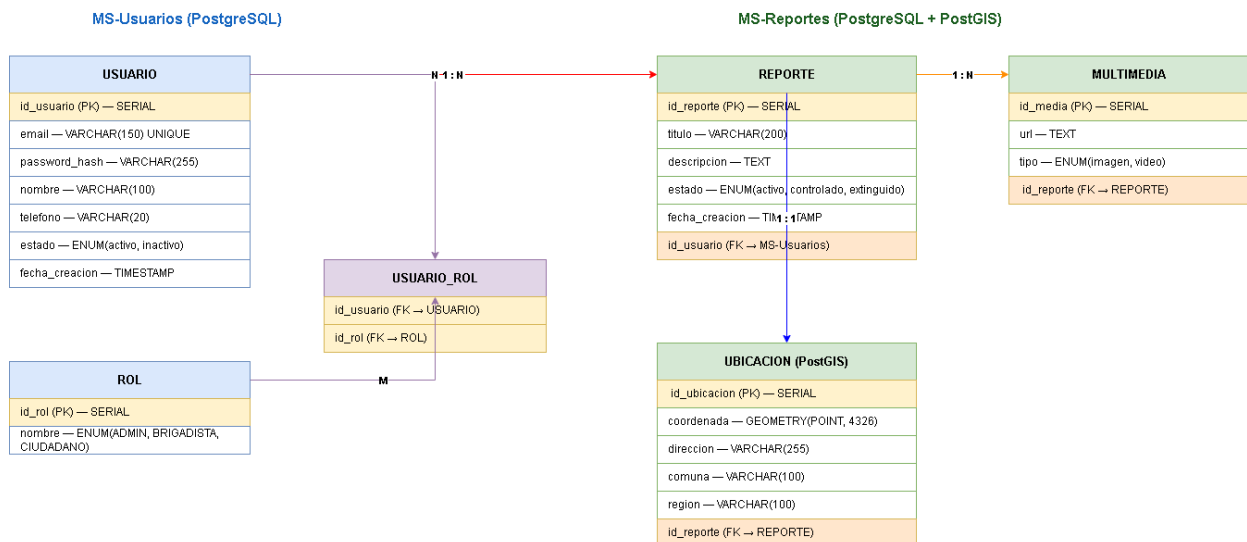
- Exponer documentación OpenAPI/Swagger en /swagger-ui.html.
- Incluye Docker Compose para ejecución local independiente del cluster Kubernetes.

6. Modelo de Datos

La estrategia de persistencia aplica el principio Database per Service: cada microservicio tiene su propia base de datos aislada en la nube. No se requieren StatefulSets ni PersistentVolumeClaims en el cluster local porque las bases de datos son serverless y gestionadas externamente.

6.1 Modelo Relacional – NeonDB PostgreSQL

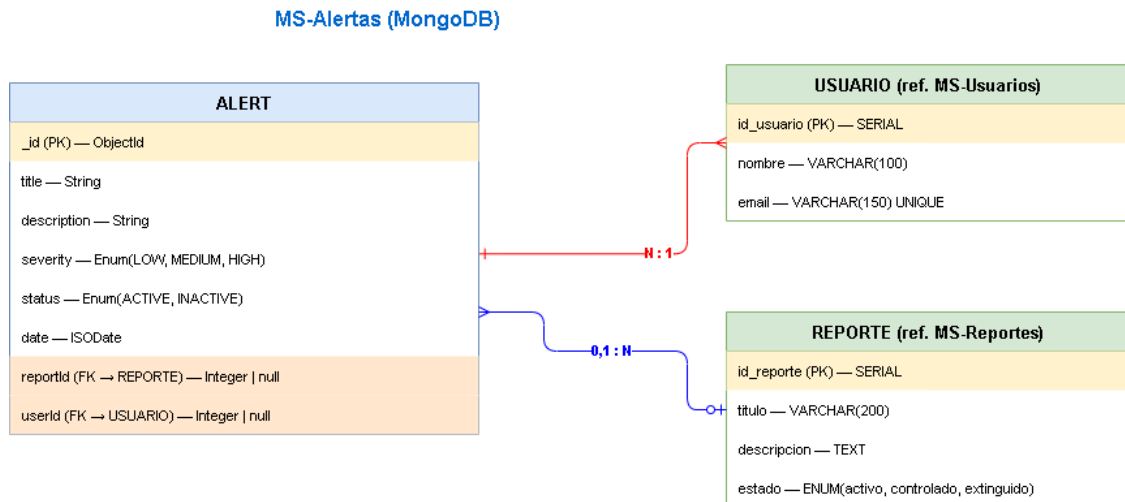
Los microservicios MS-Usuarios y MS-Reportes utilizan PostgreSQL con Spring Data JPA y migraciones Flyway. El diagrama muestra el modelo entidad-relación completo:



Las entidades principales son USUARIO (con roles via tabla pivot USUARIO_ROL), REPORTE (con coordenadas GPS gestionadas por PostGIS), UBICACION y MULTIMEDIA. La relación entre MS-Usuarios y MS-Reportes es referencial mediante id_usuario como Foreign Key lógica entre microservicios, respetando el principio Database per Service.

6.2 Modelo Documental – MongoDB Atlas

MS-Alertas utiliza MongoDB Atlas con Spring Data MongoDB. Las alertas son documentos con estructura flexible según el tipo de emergencia. La colección ALERT referencia lógicamente a USUARIO y REPORTE mediante IDs externos, sin joins entre bases de datos:



6.3 Seguridad JWT con RSA RS256 – MS-Usuarios

La autenticación implementa JWT con RSA RS256. MS-Usuarios firma los tokens con `private_key.pem` y el BFF los valida con `public_key.pem`. Las claves se montan como Kubernetes Secret (`muni-jwt-keys`) en `/secrets/` del contenedor.

Componente	Rol en la autenticación
MS-Usuarios	Firma tokens JWT con <code>private_key.pem</code> (RSA RS256)
BFF – JwtAuthFilter	Valida tokens JWT con la clave pública embebida
Kubernetes Secret	Almacena <code>private_key.pem</code> y <code>public_key.pem</code> de forma segura
Cookie HttpOnly	El BFF almacena el token en cookie no accesible desde JavaScript

7. Patrones de Diseño Aplicados

7.1 Repository Pattern

Implementado en los cuatro microservicios mediante Spring Data JPA y Spring Data MongoDB. Cada MS define una interfaz de repositorio que extiende JpaRepository o MongoRepository. Spring genera la implementación concreta automáticamente. Desacopla la lógica de negocio del acceso a datos y facilita los tests unitarios al inyectar mocks del repositorio.

7.2 Factory Method

Implementado en MS-Brigadas y MS-Alertas mediante bean factories y la anotación @Component de Spring. La fábrica recibe los parámetros del contexto (tipo de usuario, gravedad del incendio) y retorna la instancia concreta correcta sin exponer la lógica de creación al llamador. Agregar un nuevo tipo solo requiere crear una clase nueva, cumpliendo el principio Open/Closed.

7.3 Circuit Breaker – Resilience4j

Implementado en el BFF sobre las llamadas a los microservicios. Opera en tres estados: Cerrado (flujo normal), Abierto (tras fallos consecutivos devuelve respuesta de contingencia sin esperar timeout) y Semiabierto (permite una solicitud de prueba para verificar recuperación). Garantiza que el fallo de un microservicio no colapse toda la plataforma.

7.4 Singleton

En el frontend, los stores de Zustand (useAuthStore, useAlertStore, useAppStore) implementan Singleton garantizando una única instancia del estado global. En el backend, los Spring Beans (@Service, @Repository) son Singletons por defecto en el contexto de Spring.

7.5 Resumen Comparativo

Patrón	¿Qué hace?	Aplicación en el sistema	Beneficio concreto
--------	------------	--------------------------	--------------------

Repository Pattern	Separa lógica de negocio del acceso a datos.	Los 4 MS definen interfaces JpaRepository / MongoRepository.	Cambio de BD sin modificar lógica. Facilita mocks en tests.
Factory Method	Crea objetos sin if/else, delegando en fábricas.	MS-Brigadas y MS-Alertas usan fábricas Spring para instanciar tipos concretos.	Agrega tipos nuevos sin modificar código existente. Open/Closed.
Circuit Breaker	Corta el flujo ante fallos para evitar cascadas.	Resilience4j en BFF. Devuelve respuesta de contingencia ante caídas.	Fallo de un MS no colapsa la plataforma.
Singleton	Garantiza una única instancia.	Spring Beans y Zustand stores (useAuthStore, useAlertStore).	Consistencia de estado global en backend y frontend.

8. Decisiones Críticas de Diseño

Las siguientes decisiones determinaron la arquitectura y tecnología del sistema, con justificación técnica para cada una.

Decisión	Alternativa descartada	Justificación de la elección
KrakenD como API Gateway	Spring Cloud Gateway	KrakenD es stateless, escrito en Go y sin JVM. Configuración declarativa en JSON. Menor consumo de memoria y mayor rendimiento que Spring Cloud Gateway que requiere un pod Java adicional.
BFF como capa intermedia	Frontend llama directo a MS	El BFF centraliza la autenticación JWT, el manejo de cookies HttpOnly y la orquestación de llamadas múltiples. Sin BFF, el frontend conocería

		la arquitectura interna y manejaría cookies de seguridad.
JWT con RSA RS256	JWT simétrico (HS256)	Con RSA, MS-Usuarios firma con clave privada y el BFF verifica con clave pública. La clave privada nunca sale de MS-Usuarios. Con HS256, todos los servicios compartirían el mismo secreto, aumentando el riesgo de exposición.
NeonDB serverless	PostgreSQL en StatefulSet K8s local	NeonDB gestiona alta disponibilidad, backups y escalado automático. Elimina PersistentVolumeClaims y StatefulSets en el cluster local, reduciendo complejidad de despliegue.
MongoDB Atlas para alertas	PostgreSQL para alertas	Las alertas tienen estructura variable según el tipo de emergencia. El modelo documental se adapta naturalmente a documentos semiestructurados sin migraciones de esquema.
Database per Service	Base de datos compartida	Garantiza que un cambio de esquema en un MS no afecte a los demás. Cada MS puede escalar su BD de forma independiente y evolucionar sin coordinación con otros equipos.
Flyway para migraciones	Hibernate ddl-auto: create	Flyway garantiza migraciones versionadas y reproducibles. ddl-auto: create destruye datos en cada reinicio. Flyway permite rollbacks controlados y auditoría del historial de cambios de esquema.
Docker Compose + Kubernetes	Solo Docker Compose	Docker Compose para desarrollo local rápido de cada MS. Kubernetes para demostrar orquestación completa, escalado con réplicas, Secrets y ConfigMaps que reflejan un entorno de producción real.

9. Estrategia de Despliegue con Docker y Kubernetes

9.1 Docker – Construcción de Imágenes

Todos los componentes utilizan Dockerfile multi-stage: primera etapa compila (JDK 21 Alpine para Java, Node 20 Alpine para Next.js) y la segunda copia solo el artefacto compilado (JRE 21 Alpine). Reduce el tamaño de imagen aproximadamente un 60%.

9.2 Docker Compose – Ejecución Local por Microservicio

El BFF incluye un archivo docker-compose.yml único que orquesta los cinco servicios del backend (MS-Usuarios, MS-Reportes, MS-Alertas, MS-Brigadas y el propio BFF) en una sola red Docker (muni-network), permitiendo levantar el sistema completo con un solo comando sin necesitar el cluster de Kubernetes. Cada servicio se construye desde su respectivo Dockerfile mediante context apuntando a la carpeta del repositorio correspondiente. Las credenciales sensibles (contraseñas de base de datos, URI de MongoDB) se inyectan mediante variables de entorno definidas en un archivo .env excluido del repositorio con .gitignore, evitando exponerlas en GitHub. Las claves RSA de MS-Usuarios se montan como volumen de solo lectura desde el sistema de archivos local.

9.3 Kubernetes – Recursos Utilizados

Dado que las bases de datos son serverless en la nube, no se requieren StatefulSets ni PersistentVolumeClaims, simplificando considerablemente el despliegue en Docker Desktop Kubernetes.

Recurso K8s	Componentes	Justificación
Deployment	KrakenD, BFF, MS-Usuarios, MS-Reportes, MS-Alertas, MS-Brigadas, Frontend	Pods sin estado. K8s reinicia pods caídos y escala réplicas.

Service (ClusterIP)	Todos los MS y BFF	DNS interno: muni-bff:8080, ms-usuarios:8081, ms-reportes:8082, ms-alertas:8083, ms-brigadas:8084.
Service (LoadBalancer)	Frontend Next.js	Expone el frontend externamente.
ConfigMap	KrakenD (krakend.json)	Inyecta la configuración del gateway sin hardcodear en imagen.
Secret (muni-secrets)	BFF y todos los MS	Credenciales de BD y JWT almacenadas de forma segura.
Secret (muni-jwt-keys)	MS-Usuarios	Claves RSA private_key.pem y public_key.pem en /secrets/.
Namespace	muni-valle-sol	Aísla todos los recursos del proyecto en el cluster local.

9.4 Tabla Resumen de Componentes

Componente	Puerto	Réplicas	Tipo Kubernetes
KrakenD Gateway	8000	1	Deployment + ClusterIP
BFF Spring Boot	8080	2	Deployment + ClusterIP
MS-Usuarios	8081	1	Deployment + ClusterIP
MS-Reportes	8082	1	Deployment + ClusterIP
MS-Alertas	8083	1	Deployment + ClusterIP
MS-Brigadas	8084	1	Deployment + ClusterIP
Frontend Next.js	3000	2	Deployment + LoadBalancer

10. Pruebas Unitarias

Todos los componentes implementan pruebas unitarias con JUnit 5 y Mockito (backend) y Vitest con Testing Library (frontend). Cobertura mínima del 60% en todos los componentes.

10.1 Stack de Testing por Componente

Componente	Framework	Herramientas adicionales
MS-Usuarios	JUnit 5 + Mockito	Spring Boot Test, MockMvc, JaCoCo
MS-Reportes	JUnit 5 + Mockito	Spring Boot Test, MockMvc, JaCoCo
MS-Alertas	JUnit 5 + Mockito	Spring Boot Test, MockMvc, JaCoCo
MS-Brigadas	JUnit 5 + Mockito	Spring Boot Test, MockMvc, JaCoCo
BFF	JUnit 5 + Mockito	Spring Boot Test, WebTestClient, JaCoCo
Frontend Next.js	Vitest	Testing Library, @testing-library/react, jsdom

10.2 Cobertura de Pruebas por Componente

Componente	Clases con tests	Cobertura	Capas cubiertas
BFF	AlertController, BrigadaController, ReportController, AlertService, BrigadaService, ReportService, JwtAuthFilter, GlobalExceptionHandler	> 60%	Controller, Service, Security, Exception
MS-Usuarios	AuthService, JwtService, UserService, UserController, SecurityConfig	> 60%	Controller, Service, Security
MS-Reportes	ReporteService, ReporteController, ReporteRepository (mock)	> 60%	Controller, Service

MS-Alertas	AlertaService, AlertaController, AlertaRepository (mock)	> 60%	Controller, Service
MS-Brigadas	BrigadaService, BrigadaController, BrigadaRepository (mock)	> 60%	Controller, Service
Frontend	AlertPanel, ReportTable, Footer, alertService, authService, useAuthStore, useAlertStore, useAppStore	> 60%	Components, Services, Stores

10.3 Ejemplos de Pruebas Implementadas

BFF – AlertControllerTest (JUnit 5 + Mockito):

```
@Test
void getAlerts_returnsListSuccessfully() {
    when(alertService.getAlerts()).thenReturn(List.of(alertDTO));
    ResponseEntity<?> response = alertController.getAlerts(mockCookie);
    assertEquals(200, response.getStatusCode().value());
}
```

Frontend – alertService.test.ts (Vitest):

```
it('should fetch alerts successfully', async () => {
    global.fetch = vi.fn().mockResolvedValue({ ok: true, json: () => [alert]
});
    const result = await getAlerts();
    expect(result).toEqual([alert]);
});
```

10.4 Cómo Ejecutar las Pruebas

Backend – cualquier microservicio o BFF:

```
./mvnw test # Ejecutar todos los tests
./mvnw test jacoco:report # Reporte en target/site/jacoco/index.html
```

Frontend – Next.js:

```
npm test # Ejecutar todos los tests
npm run test:coverage # Generar reporte de cobertura
```

11. Diagrama de Arquitectura

El diagrama muestra el flujo de comunicación completo: Frontend Next.js → KrakenD API Gateway → BFF Spring Boot → 4 Microservicios → NeonDB / MongoDB Atlas. Se ilustra el namespace muni-valle-sol en Docker Desktop Kubernetes, los Secrets y ConfigMaps.

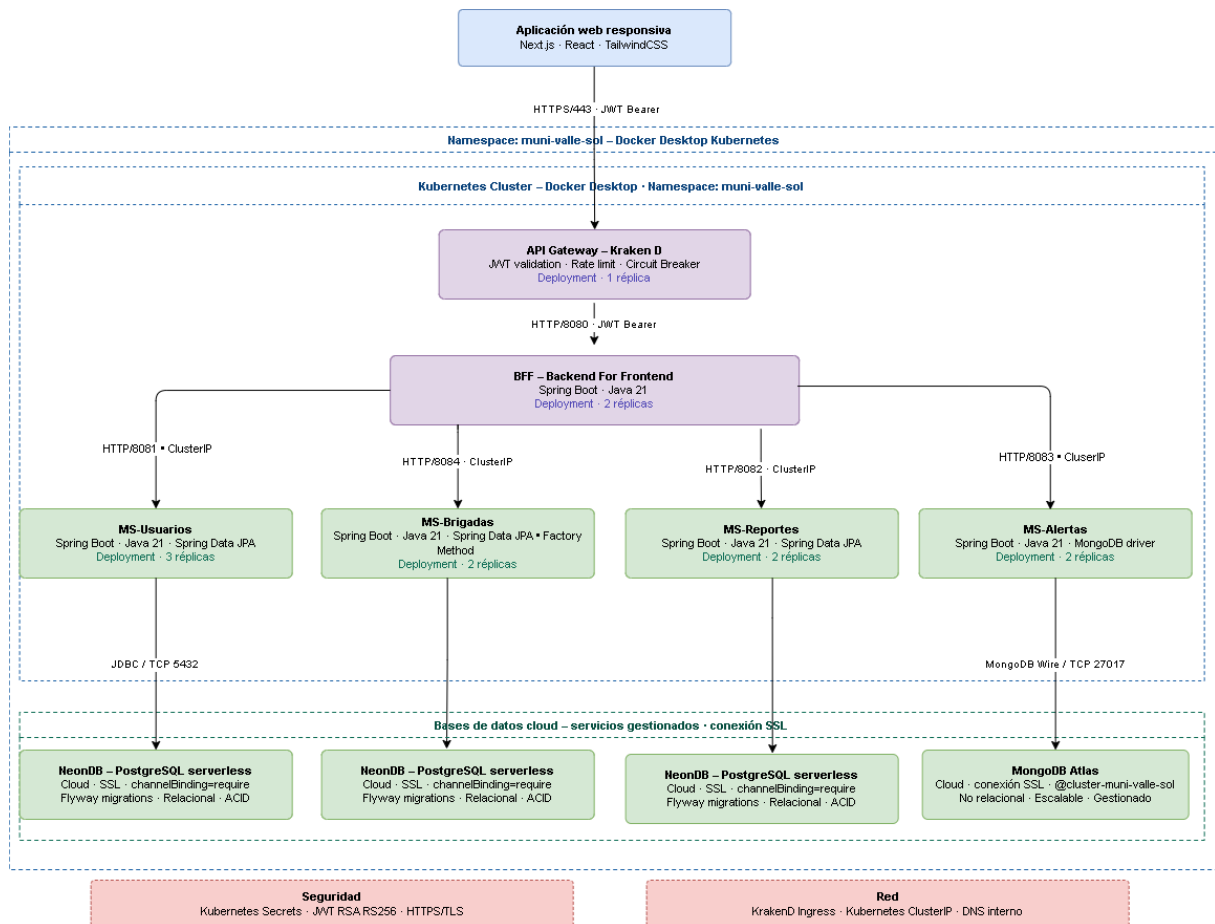


Diagrama C1 – Contexto del Sistema

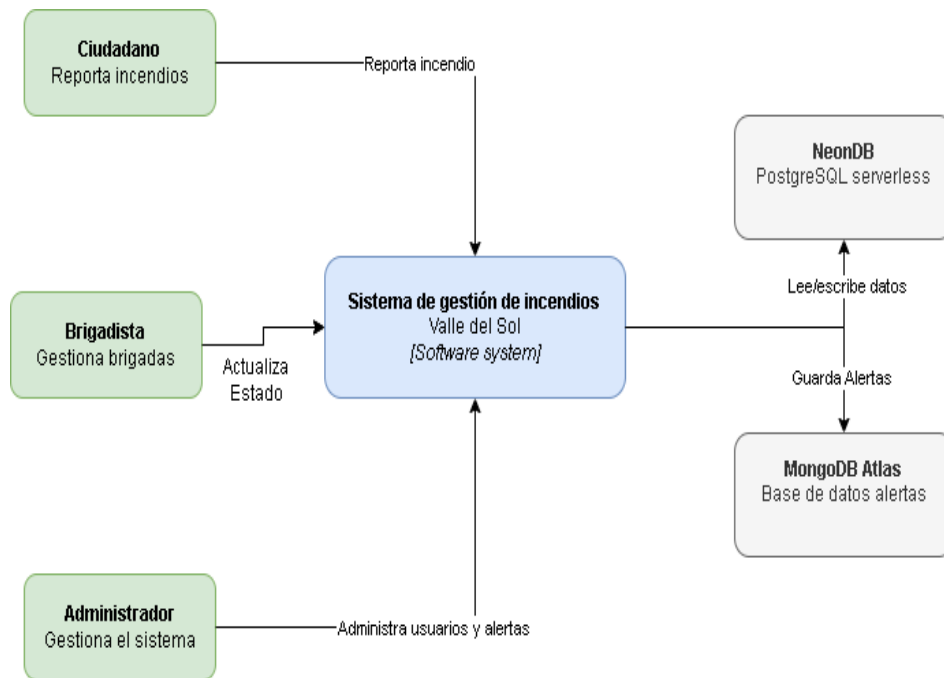


Diagrama C2 – Arquitectura de Contenedores

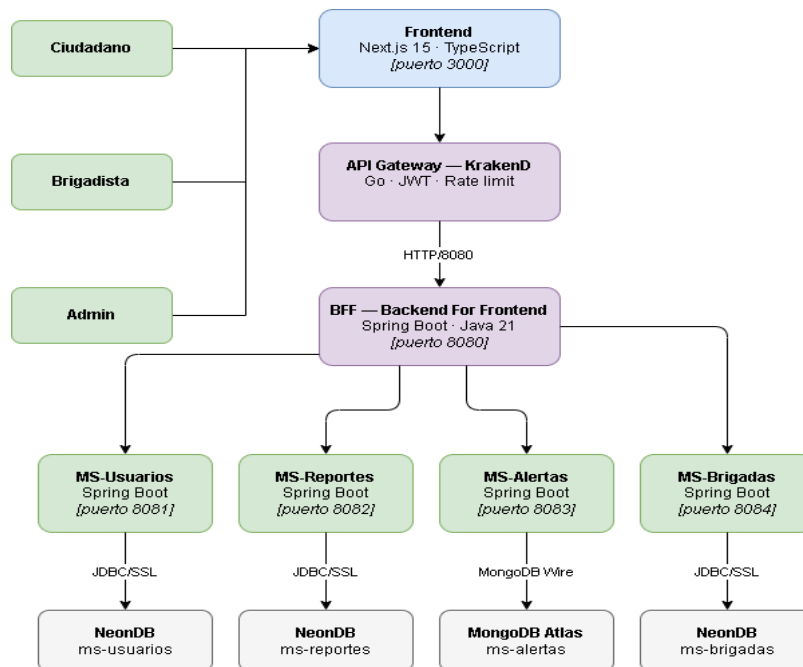
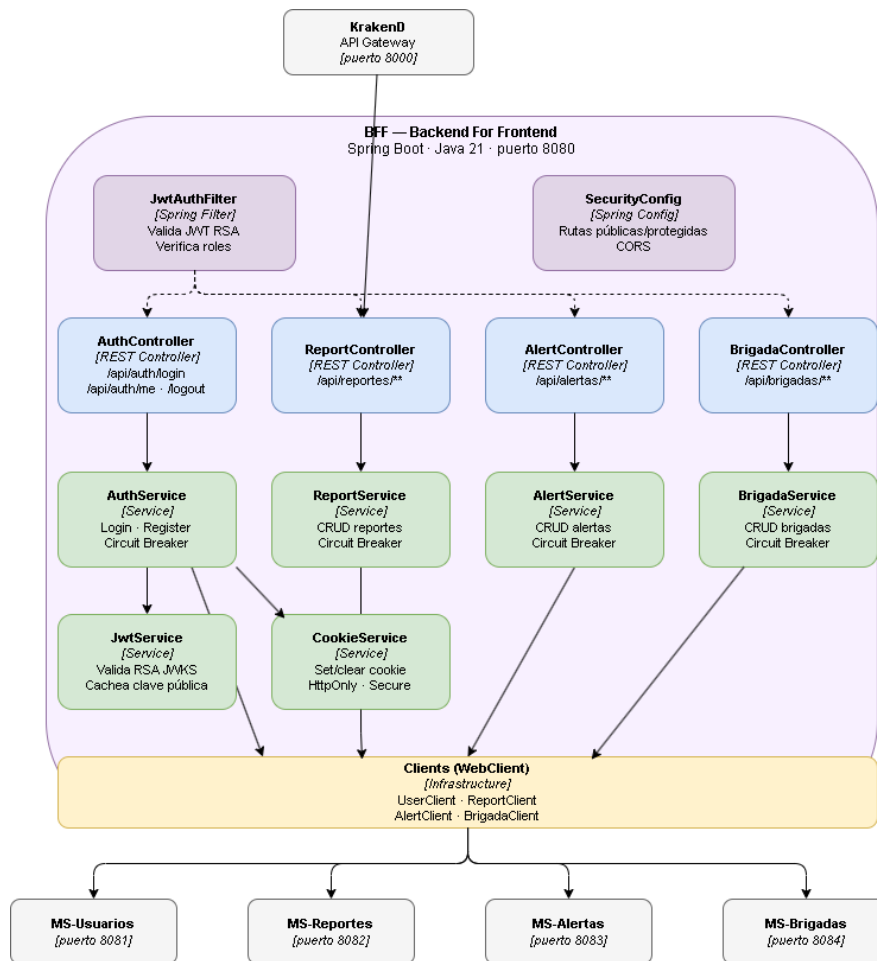


Diagrama C3 – Componentes



12. Consideraciones Éticas: Seguridad, Privacidad y Sostenibilidad

12.1 Seguridad

La arquitectura implementa seguridad en capas. El BFF centraliza la autenticación mediante JWT RS256, almacenando el token en una cookie HttpOnly protegiendo contra XSS. Spring Security en MS-Usuarios gestiona los roles (CIUDADANO, BRIGADISTA, ADMIN). KrakenD propaga los headers de autenticación sin exponerlos al exterior.

12.2 Privacidad

Los datos personales (ubicación GPS, identidad, historial) se almacenan en bases de datos separadas por microservicio. El acceso a cada BD está restringido al microservicio correspondiente mediante Kubernetes Service DNS. Las credenciales y claves JWT se almacenan como Kubernetes Secrets, nunca hardcodeadas en el código ni en los repositorios.

12.3 Sostenibilidad

El uso de bases de datos serverless elimina la gestión de infraestructura de almacenamiento persistente en el cluster local. Kubernetes permite escalar los microservicios durante emergencias y reducir réplicas en períodos de baja demanda. La arquitectura modular permite reemplazar componentes sin reescribir el sistema completo.

13. Repositorios GitHub

Todos los repositorios incluyen README.md con instrucciones de instalación, ejecución y pruebas, y docker-compose.yml para ejecución local.

Repositorio	Descripción
github.com/JOAKOO123/muni-valle-sol-bff	BFF Spring Boot – JWT, orquestación de MS, CORS, Swagger, Docker Compose
github.com/JOAKOO123/muni-valle-sol-frontend	Frontend Next.js 15 – MapLibre GL, Zustand, TypeScript, Vitest
github.com/lsn33/muni-valle-sol-ms-usuarios	MS-Usuarios – Spring Security, JWT RS256, Flyway, NeonDB, Docker Compose
github.com/lsn33/muni-valle-sol-ms-reportes	MS-Reportes – JPA, PostGIS, Flyway, NeonDB, Docker Compose
github.com/JOAKOO123/muni-valle-sol-ms-alertas	MS-Alertas – MongoDB Atlas, Factory Method, Docker Compose
github.com/JOAKOO123/muni-valle-sol-ms-brigadas	MS-Brigadas – Factory Method, Flyway, NeonDB, Docker Compose

14. Conclusión

La solución implementada aborda de manera directa y técnicamente justificada las problemáticas en la gestión de incendios de la Municipalidad Valle del Sol. La arquitectura de cuatro microservicios independientes, un BFF, un API Gateway KrakenD y un frontend Next.js, orquestados con Docker y Kubernetes, proporciona una plataforma escalable, resiliente y con persistencia garantizada mediante bases de datos serverless en la nube.

Los patrones de diseño aplicados (Repository Pattern, Factory Method, Circuit Breaker, Singleton) aseguran mantenibilidad, extensibilidad y tolerancia a fallos. Las decisiones críticas de diseño, como el uso de KrakenD en lugar de Spring Cloud Gateway, JWT con RSA RS256 y Database per Service, fueron tomadas con criterios técnicos claros que minimizan riesgos y complejidad operacional.

La implementación de pruebas unitarias con JUnit 5, Mockito y Vitest alcanza coberturas superiores al 60% en todos los componentes, validando la calidad del código. El uso de Docker Compose por microservicio facilita el desarrollo local, mientras que Kubernetes demuestra la capacidad de orquestación y escalado del sistema en un entorno que replica condiciones de producción.